Software Process Models

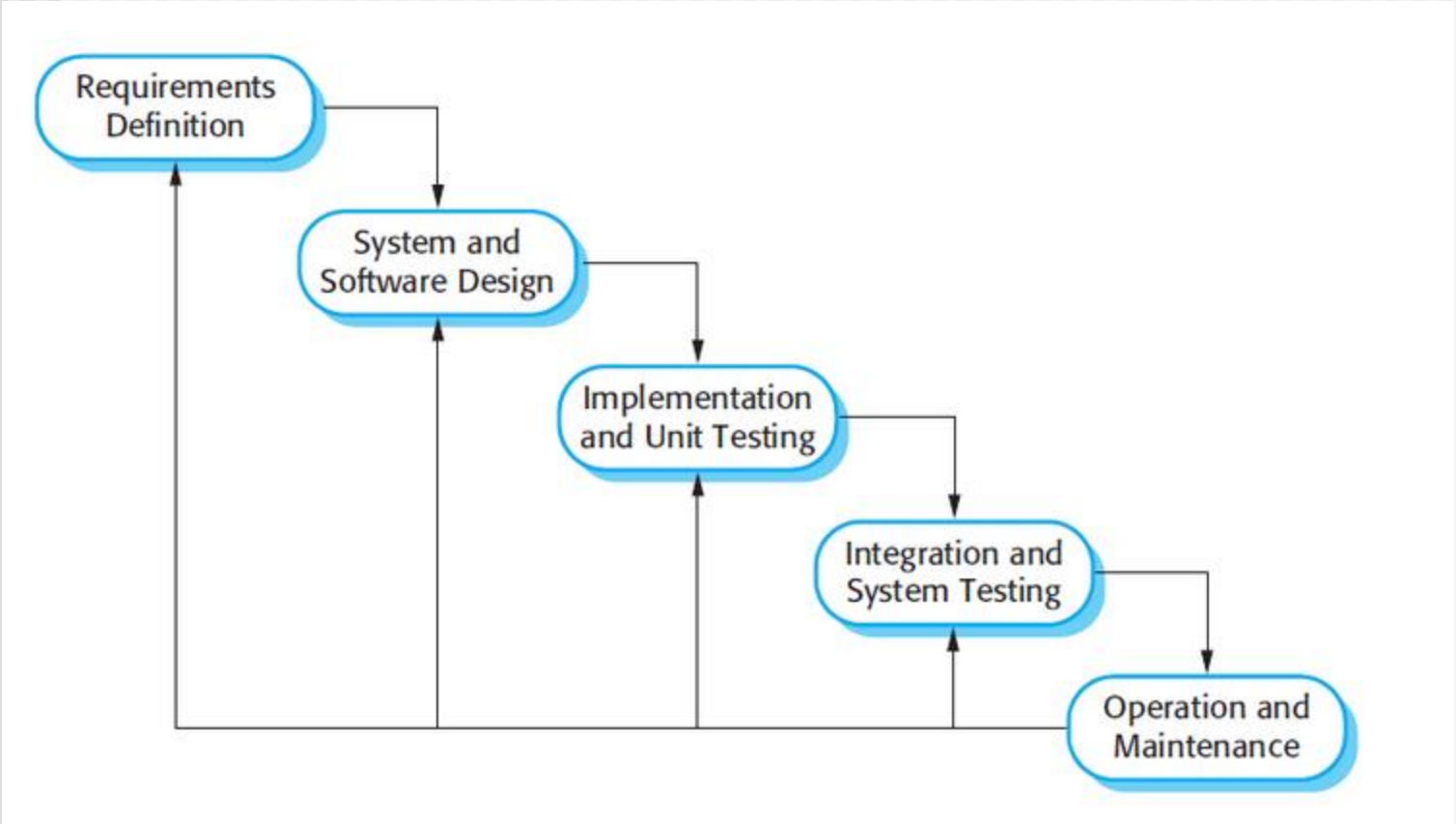
Week 3

Chanduni Gamage
Lecturer (Probationary)
Dept. of ICT
Faculty of Technology
University of Ruhuna

The Waterfall Model

- The Waterfall Model was the first Process Model to be introduced.
- It is also referred to as a **linear-sequential life cycle model**.
- It is very simple to understand and use.
- In a waterfall model, each phase must be completed before the next phase can begin and there is no overlapping in the phases.
- The Waterfall model is the earliest SDLC approach that was used for software development.

The Waterfall Model



Waterfall Model Phases

- There are separate identified phases in the waterfall model:
  - Requirements analysis and definition
  - System and software design
  - Implementation and unit testing
  - Integration and system testing
  - Operation and maintenance
- The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway. In principle, a phase has to be complete before moving onto the next phase.

Phases in Waterfall model

- **Requirement Gathering and analysis**
- All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- The system's services, constraints and goals are established by consultation with system users. They are then defined in a manner that is understandable by both users and development staff.
  - This phase can be divided into
    - Feasibility Study (Often carried out separately)
    - Requirements Analysis
    - Requirements definition
    - Requirements specification
  - Deliverables: SRS, Project Plan(Milestones, Financial Plan), Test Plan, Quality Manuals

Phases in Waterfall model

- **System Design** − The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
  - Deliverables: Design Docs, Unit Test docs

- System Design: Partition the requirements to hardware or software systems. Establishes an overall system architecture
- Software Design: Represent the software system functions in a form that can be transformed into one or more executable programs

Phases in Waterfall model

- **Implementation** − With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality against specifications, which is referred to as ***Unit Testing***.
  - Deliverables: Source code, Operation Guides
- **Integration and Testing** − All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.

Phases in Waterfall model

- **Deployment of system/Operation** − Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
  - Deliverables: Maintenance Plan
- **Maintenance** − There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

Waterfall model problems

- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
  - Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.
  - Few business systems have stable requirements.
- The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites.
  - In those circumstances, the plan-driven nature of the waterfall model helps coordinate the work.

Discussion of the Waterfall Model

- Advantages:

This method is good when requirements are clear and stable

  - Process visibility
  - Dependence on individuals
  - Quality control
  - Cost control

- Disadvantages:

Each stage in the process reveals new understanding of the previous stages, that requires the earlier stages to be revised.

Waterfall Model - Advantages

- Simple and easy to understand and use
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Clearly defined stages.
- Well understood milestones.
- Easy to arrange tasks.
- Process and results are well documented.

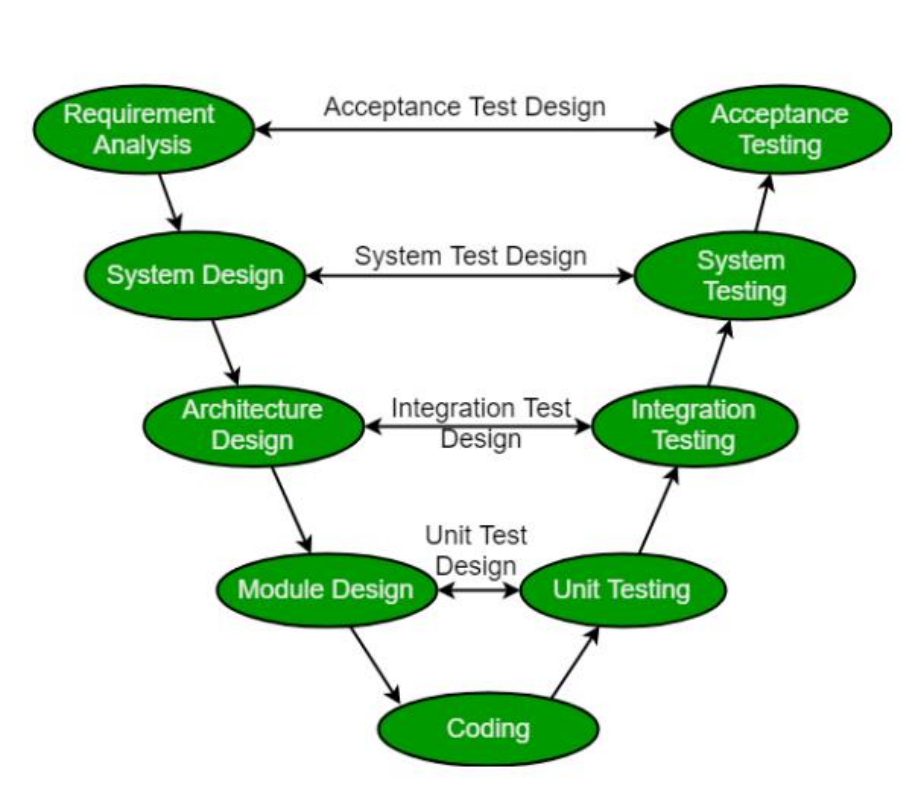
Waterfall Model - Disadvantages

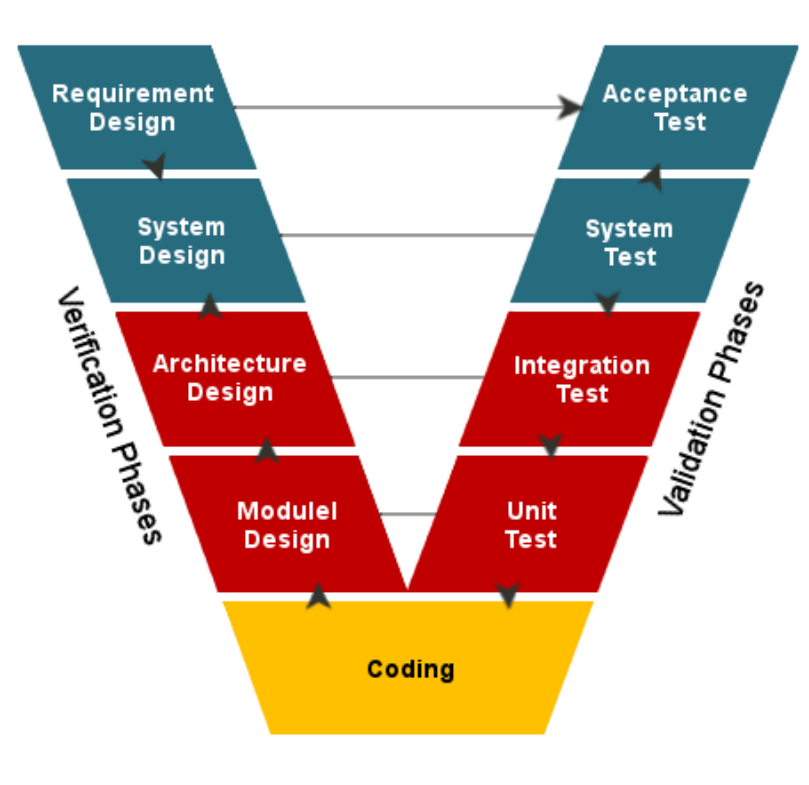
- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty.
- Not a good model for complex projects.
- Poor model for long and ongoing projects.
- Not suitable for projects where requirements are at a moderate to high risk of changing. So, risk and uncertainty is high with this process model.
- It is difficult to measure progress within stages.
- Cannot accommodate changing requirements.
- Difficult and expensive to make changes to documents,
- Adjusting scope during the life cycle can end a project.
- Integration is done as a “big bang”, at the very end, which doesn’t allow for identifying any technological or business bottleneck or challenges early.

The V-Model

- Under the V-Model, the corresponding testing phase of the development phase is planned in parallel.
- So, there are Verification phases on one side of the ‘V’ and Validation phases on the other side.
- The Coding Phase joins the two sides of the V-Model.

The V-Model


Requirement Analysis
Acceptance Test Design
Acceptance Testing
System Design
System Test Design
System Testing
Architecture Design
Integration Test Design
Integration Testing
Unit Test Design
Module Design
Unit Testing
Coding


Requirement Design
Acceptance Test
System Design
System Test
Architecture Design
Integration Test
Modulel Design
Unit Test
Coding
Verification Phases
Validation Phases

V-Model - Verification Phases

- **Business Requirement Analysis**
  - This is the first phase in the development cycle where the product requirements are understood from the customer’s perspective.
  - This phase involves detailed communication with the customer to understand his expectations and exact requirement.
  - This is a very important activity and needs to be managed well, as most of the customers are not sure about what exactly they need.
  - The **acceptance test design planning** is done at this stage as business requirements can be used as an input for acceptance testing.

V-Model - Verification Phases

- **System Design**
  - Once you have the clear and detailed product requirements, it is time to design the complete system.
  - The system design will have the understanding and detailing the complete hardware and communication setup for the product under development.
  - The system test plan is developed based on the system design.
  - Doing this at an earlier stage leaves more time for the actual test execution later.

V-Model - Verification Phases

- **Architectural Design**
  - Architectural specifications are understood and designed in this phase.
  - Usually more than one technical approach is proposed and based on the technical and financial feasibility the final decision is taken.
  - The system design is broken down further into modules taking up different functionality. This is also referred to as **High Level Design (HLD)**.
  - The data transfer and communication between the internal modules and with the outside world (other systems) is clearly understood and defined in this stage. With this information, integration tests can be designed and documented during this stage.

V-Model - Verification Phases

- **Module Design**
  - In this phase, the detailed internal design for all the system modules is specified, referred to as **Low Level Design (LLD)**.
  - It is important that the design is compatible with the other modules in the system architecture and the other external systems.
  - The unit tests are an essential part of any development process and helps eliminate the maximum faults and errors at a very early stage.
  - These unit tests can be designed at this stage based on the internal module designs.

V-Model - Coding Phase

- The actual coding of the system modules designed in the design phase is taken up in the Coding phase.
- The best suitable programming language is decided based on the system and architectural requirements.
- The coding is performed based on the coding guidelines and standards.
- The code goes through numerous code reviews and is optimized for best performance before the final build is checked into the repository.

In-Class Activity 01

- Describe the following types of software testing
  - Unit testing
  - Integration testing
  - System testing
  - Acceptance testing

V-Model - Validation Phases

- **Unit Testing**
  - Unit tests designed in the module design phase are executed on the code during this validation phase. Unit testing is the testing at code level and helps eliminate bugs at an early stage, though all defects cannot be uncovered by unit testing.
- **Integration Testing**
  - Integration testing is associated with the architectural design phase. Integration tests are performed to test the coexistence and communication of the internal modules within the system.

V-Model - Validation Phases

- **System Testing**
  - System testing is directly associated with the system design phase. System tests check the entire system functionality and the communication of the system under development with external systems. Most of the software and hardware compatibility issues can be uncovered during this system test execution.
- **Acceptance Testing**
  - Acceptance testing is associated with the business requirement analysis phase and involves testing the product in user environment. Acceptance tests uncover the compatibility issues with the other systems available in the user environment. It also discovers the non-functional issues such as load and performance defects in the actual user environment.

V-Model - Advantages

- This is a highly-disciplined model and Phases are completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Simple and easy to understand and use.
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.

V-Model - Disadvantages

- High risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- Not suitable for the projects where requirements are at a moderate to high risk of changing.
- Once an application is in the testing stage, it is difficult to go back and change a functionality.
- No working software is produced until late during the life cycle.

Evolutionary Development

Type 1: Exploratory development

- Objective is to work with customers and to evolve a final system from an initial outline specification. Should start with well-understood requirements

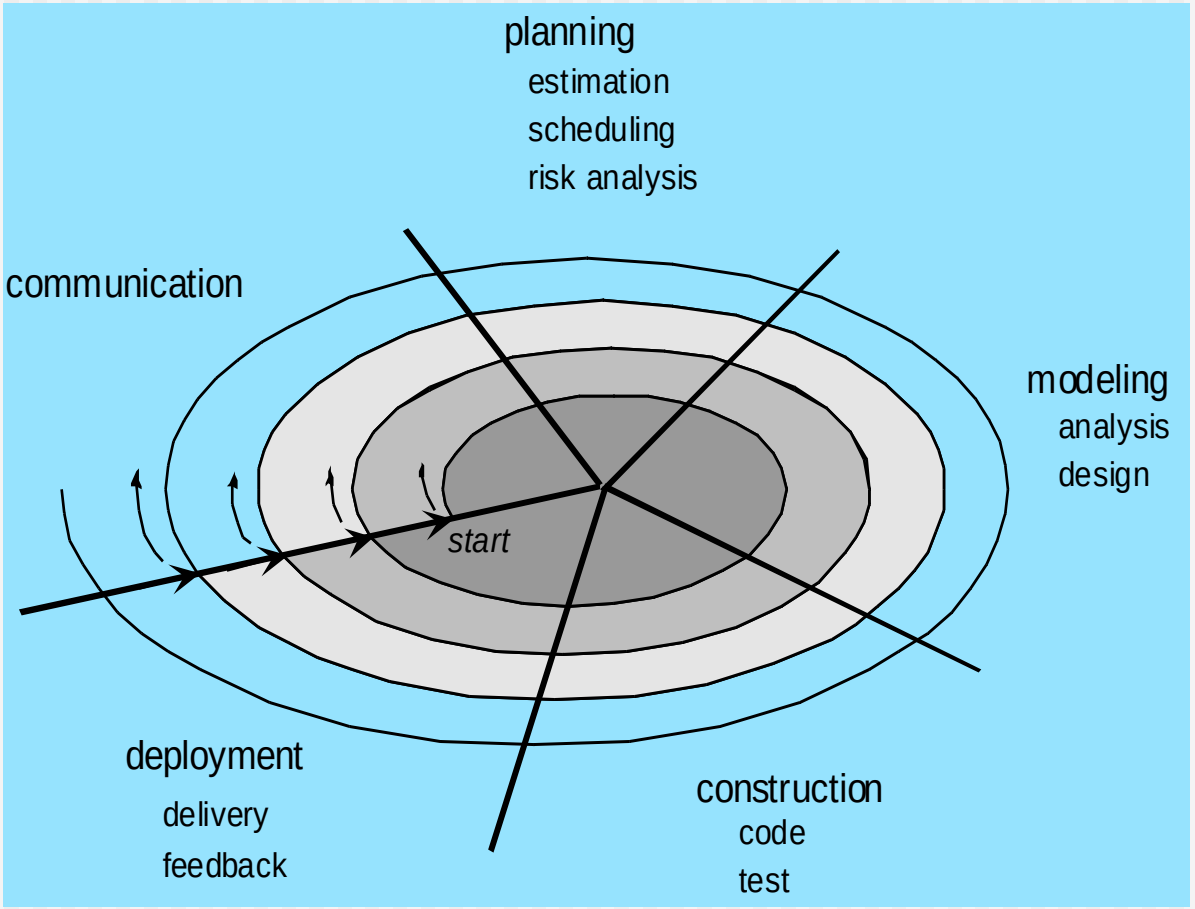
  *Customer assisted development that **evolves a product***

Type 2: Throw-away prototyping

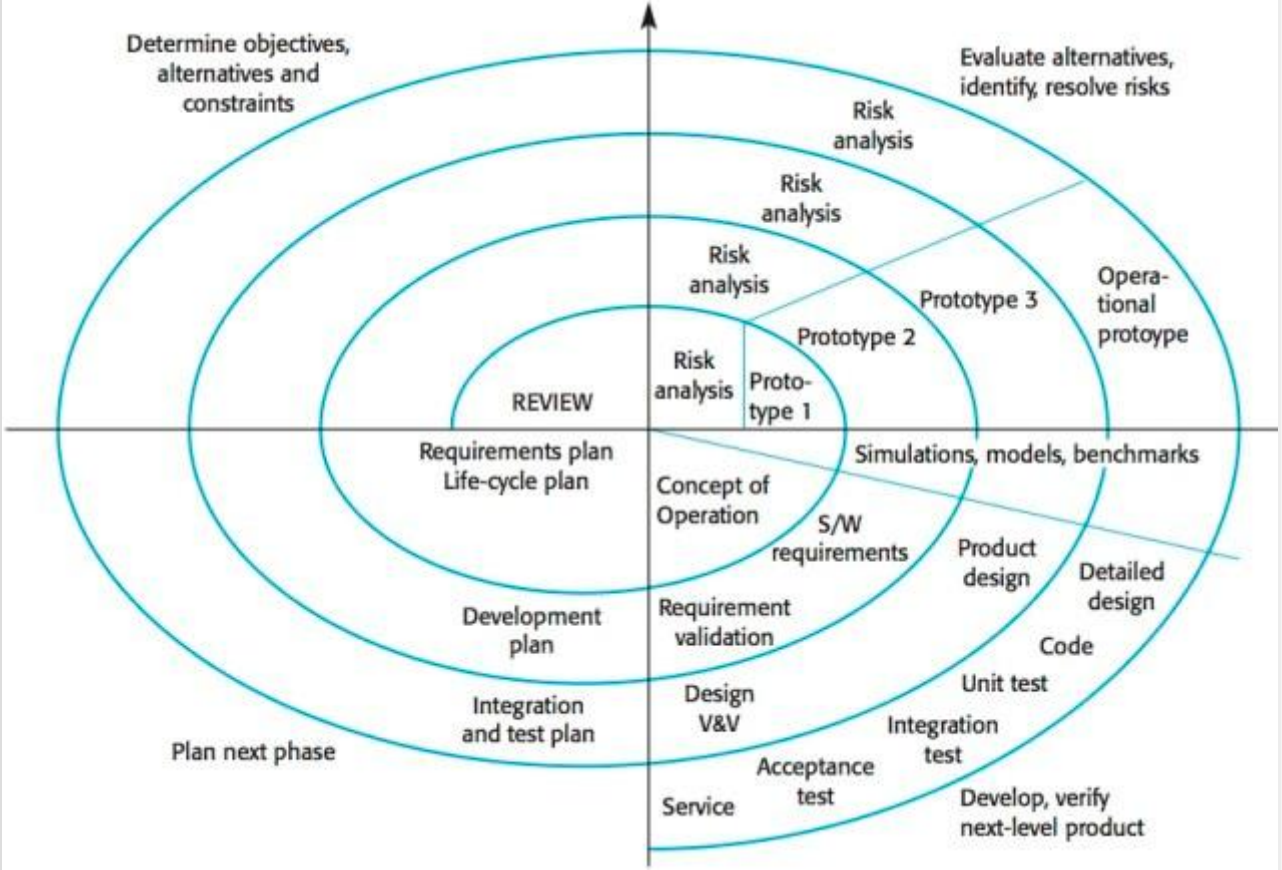
- Objective is to understand the system requirements. Should start with poorly understood requirements

  *Customer assisted development that **evolves requirements***

Evolutionary Models Type 1 : The Spiral



Boehm's spiral model of the software process



Boehm's Spiral Model

- The spiral model is a risk-driven where the process is represented as spiral rather than a sequence of activities.
- It was designed to include the best features from the waterfall and prototyping models, and introduces a new component; **risk-assessment**.
- Each loop in the spiral represents a phase in the process.
- No fixed phases such as specification or design-loops in the spiral are chosen depending on what is required.
- Risks are explicitly assessed and resolved throughout the process.

Spiral Model Sectors

- Objective setting
  - Specific objectives for the phase are identified.
- Risk assessment and reduction
  - Risks are assessed and activities put in place to reduce the key risks.
- Development and validation
  - A development model for the system is chosen which can be any of the generic models.
- Planning
  - The project is reviewed and the next phase of the spiral is planned and a decision is made whether to continue with a further loop or not.

The Spiral Model - Advantages

- Realism: the model accurately reflects the iterative nature of software development on projects with unclear requirements
- Flexible: incorporates the advantages of the waterfall and evolutionary methods
- Comprehensive model decreases risk
- Good project visibility.

The Spiral Model - Disadvantages

- Needs technical expertise in risk analysis and risk management to work well.
- Model is poorly understood by non-technical management, hence not so widely used
- Complicated model, needs competent professional management. High administrative overhead

Evolutionary Models Type 1 : Agile Software Processes

- **Agile software development** is a conceptual framework for software engineering that promotes development iterations throughout the life-cycle of the project.
- Software developed during one unit of time is referred to as an iteration, which may last from one to four weeks.
- Agile methods also emphasize working software as the primary measure of progress
- Existing Agile Methods
  - Extreme Programming (“XP”)
  - Agile Unified Process
  - Scrum

Software Prototyping

- A prototype is an initial version of a system used to demonstrate concepts and try out design options.
- A prototype can be used in:
  - The requirements engineering process to help with requirements elicitation and validation;
  - In design processes to explore options and develop a UI design;
  - In the testing process to run back-to-back tests.

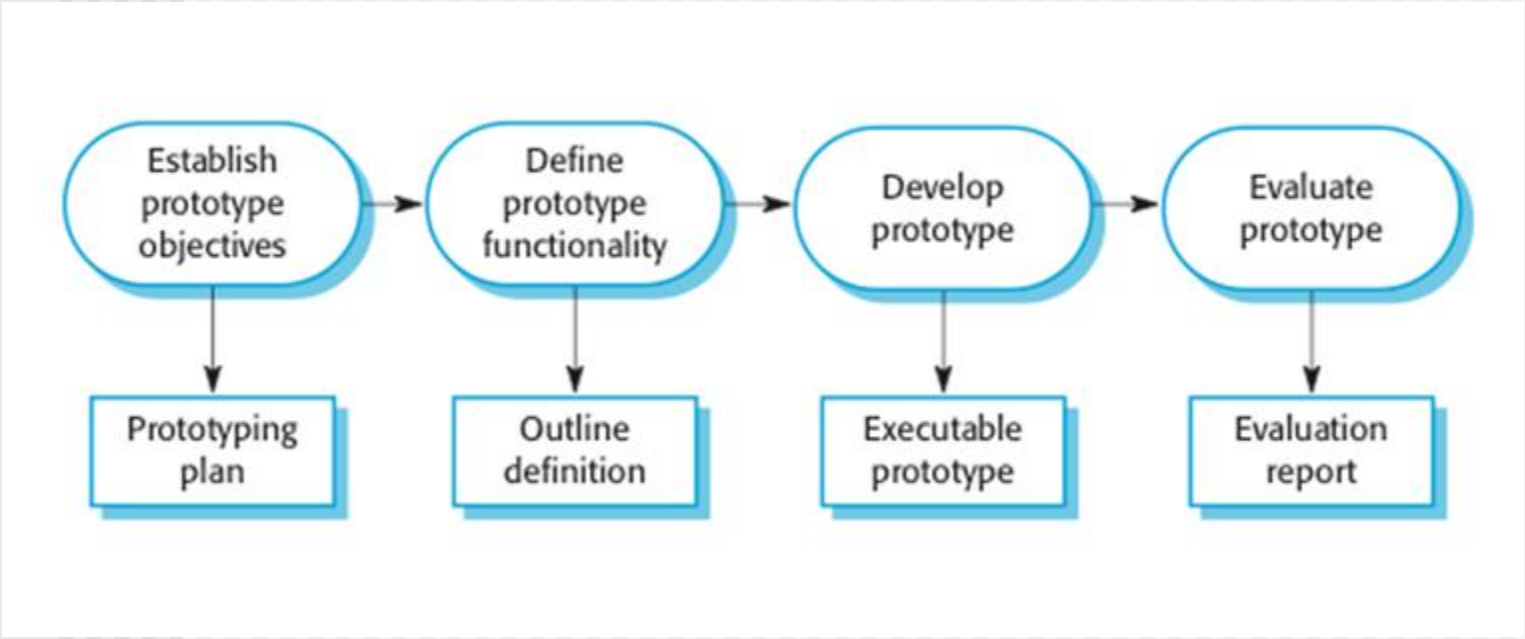
Benefits of Prototyping

- Improved system usability.
- A closer match to users' real needs.
- Improved design quality.
- Improved maintainability.
- Reduced development effort.

Other considerations:

- The prototype may be a usable program but is not suitable as the final software product
- The code for the prototype is thrown away. However experience gathered helps in developing the actual system.
- The development of a prototype might involve extra cost, but overall cost might turnout to be lower than that of an equivalent system development using the waterfall method

The process of prototype development



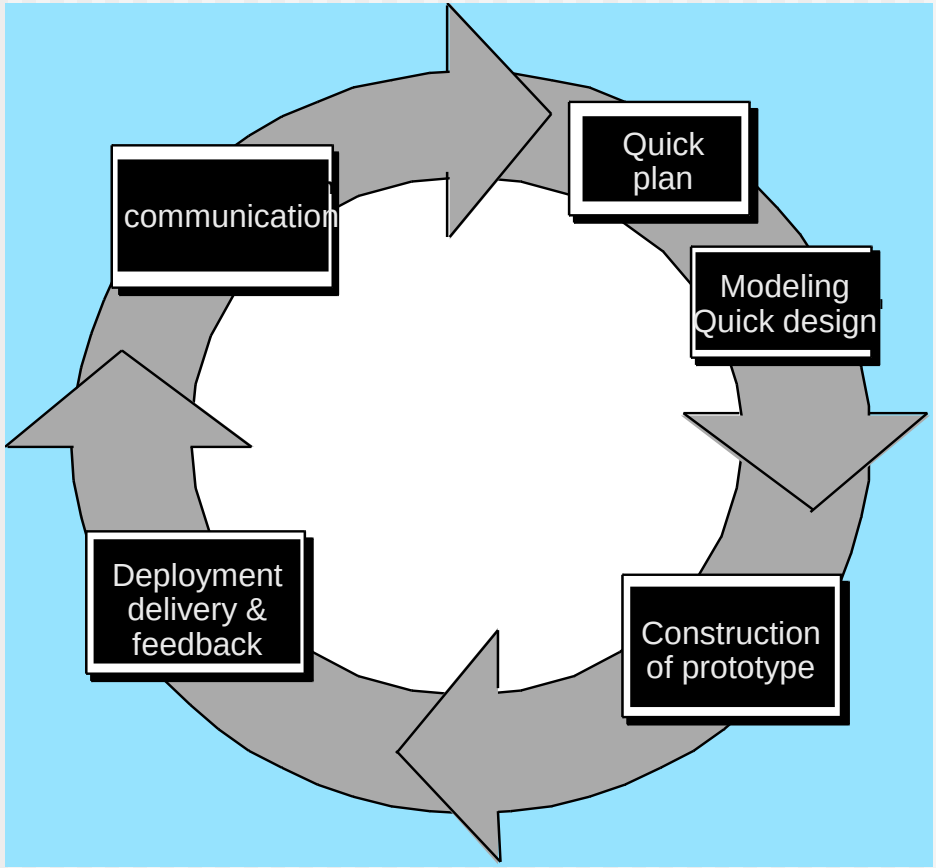
Prototype development

- May be based on rapid prototyping languages or tools
- May involve leaving out functionality
  - Prototype should focus on areas of the product that are not well-understood;
  - Error checking and recovery may not be included in the prototype;
  - Focus on functional rather than non-functional requirements such as reliability and security

Throw-away prototypes

- Prototypes should be discarded after development as they are not a good basis for a production system:
  - It may be impossible to tune the system to meet non-functional requirements;
  - Prototypes are normally undocumented;
  - The prototype structure is usually degraded through rapid change;
  - The prototype probably will not meet normal organisational quality standards.

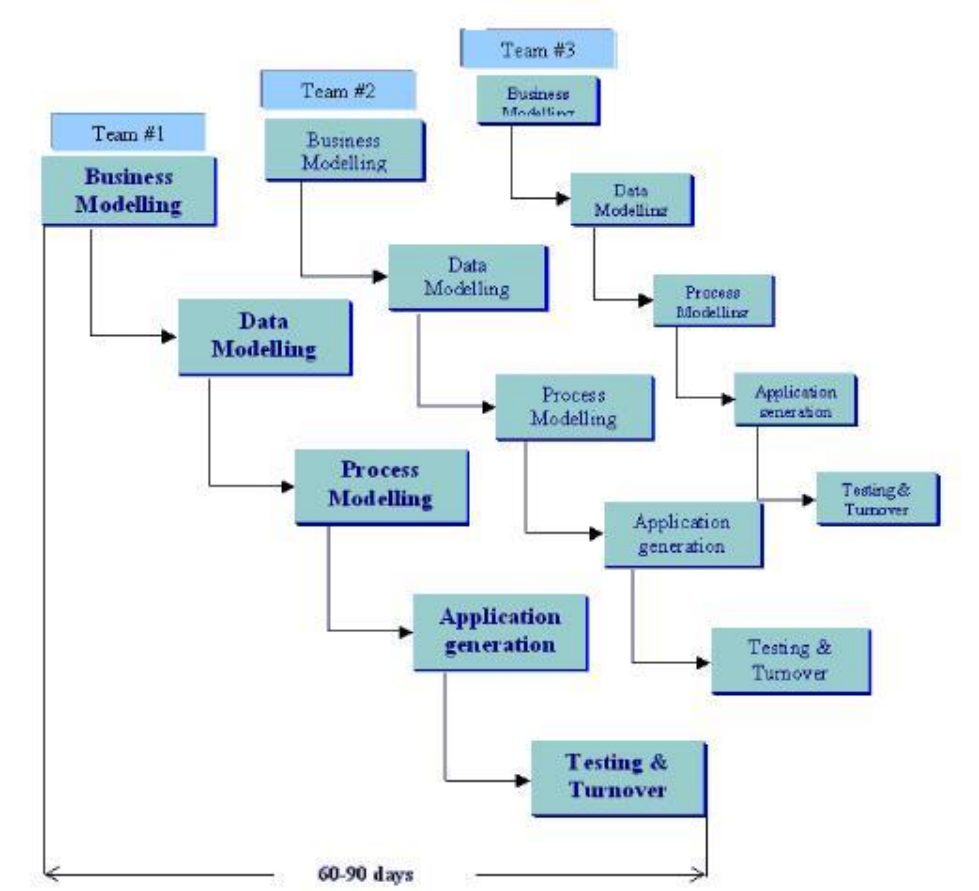
Evolutionary Models Type 2 : Prototyping



Rapid Application Development (RAD)

- Developed by IBM in 1980
- User Participation is essential
- Build a rapid prototype
- Give it to user for evaluation and obtain feedback
- Prototype is refined
- Emphasizes extensive user involvement in the rapid and evolutionary construction of working prototypes of a system to accelerate the systems development process

Rapid Application Development (RAD)

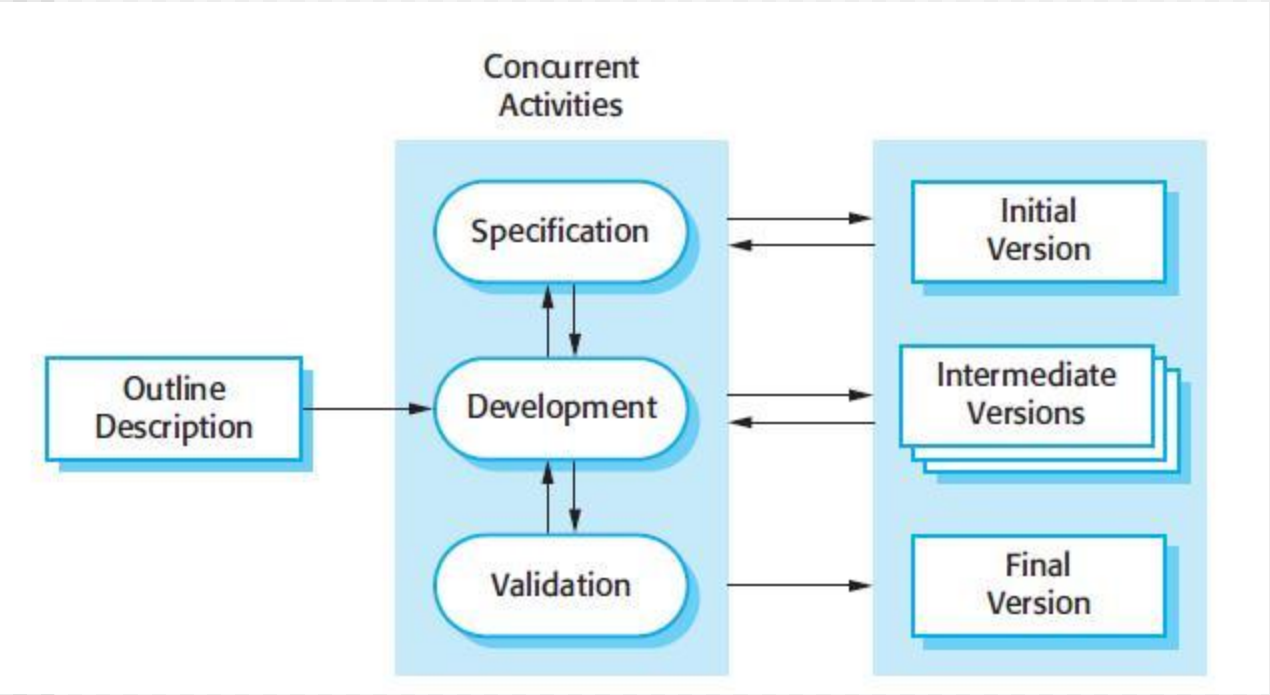


RAD Features

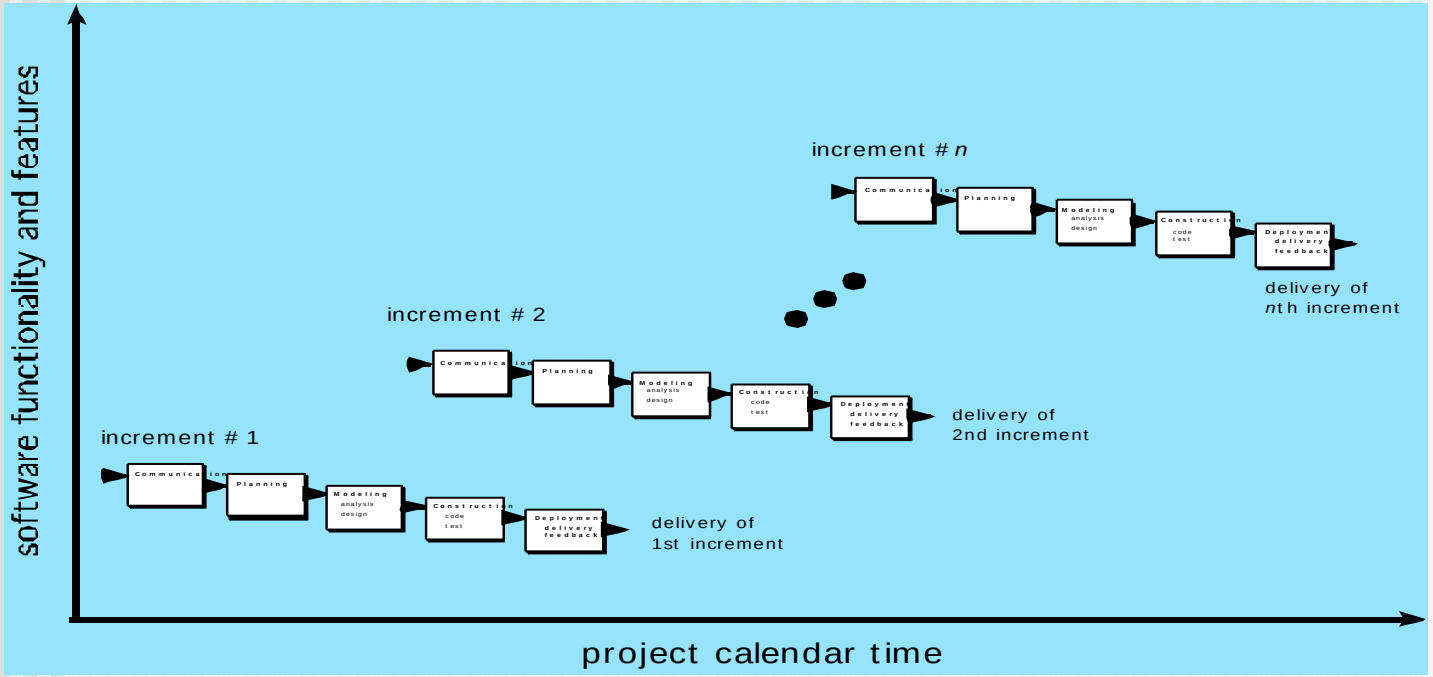
- Not an appropriate model in the absence of user participation
- Reusable components are required to reduce development time
- Highly specialized & skilled developers are required and such developers are not easily available.

The Incremental Model

Incremental development is based on the idea of developing an initial implementation, exposing this to user feedback, and evolving it through several versions until an acceptable system has been developed.



The Incremental Model



This model combines the elements of the waterfall model with the iterative philosophy of prototyping.

Incremental Model - Advantages

- Generates working software quickly and early during the software life cycle.
- More flexible – less costly to change scope and requirements.
- Easier to test and debug during a smaller iteration.
- Easier to manage risk because risky pieces are identified and handled during its iteration.
- Each iteration is an easily managed milestone.

Incremental Model - Advantages

- The cost of accommodating changing customer requirements is reduced.
  - The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
- It is easier to get customer feedback on the development work that has been done.
  - Customers can comment on demonstrations of the software and see how much has been implemented.
- More rapid delivery and deployment of useful software to the customer is possible.
  - Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Incremental Model - Disadvantages

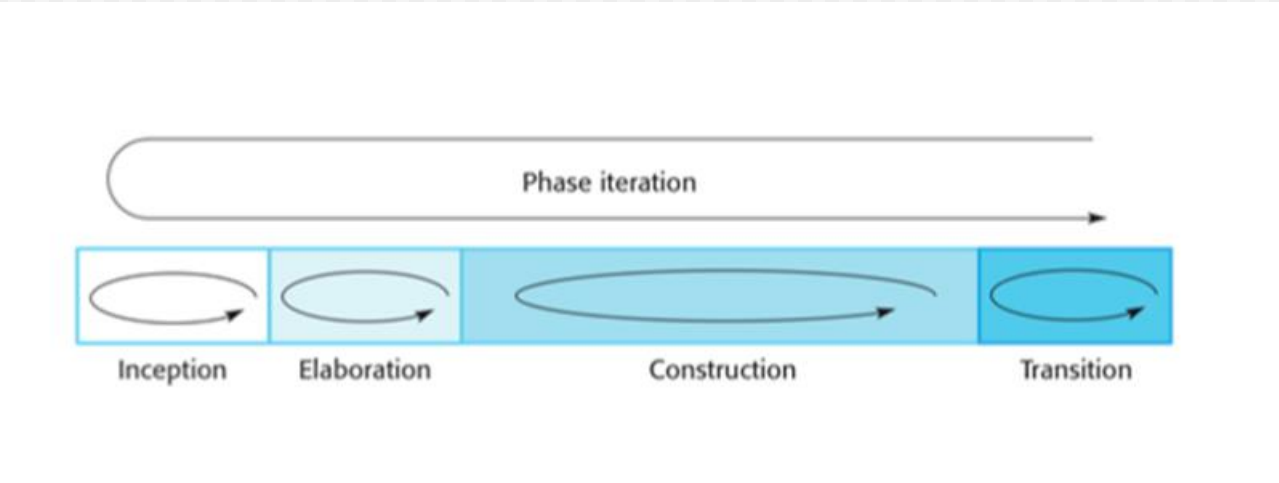
- Each phase of an iteration is rigid and do not overlap each other.
- Problems may arise pertaining to system architecture because not all requirements are gathered up front for the entire software life cycle.
- The process is not visible.
  - Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.

Incremental Model - Disadvantages

- System structure tends to degrade as new increments are added.
  - Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

Iterative Development

- Iterative development model aims to develop a system through building small portions of all the features, across all components.
- In each increment, a slice of system features is delivered, passing through the requirements till the deployment.



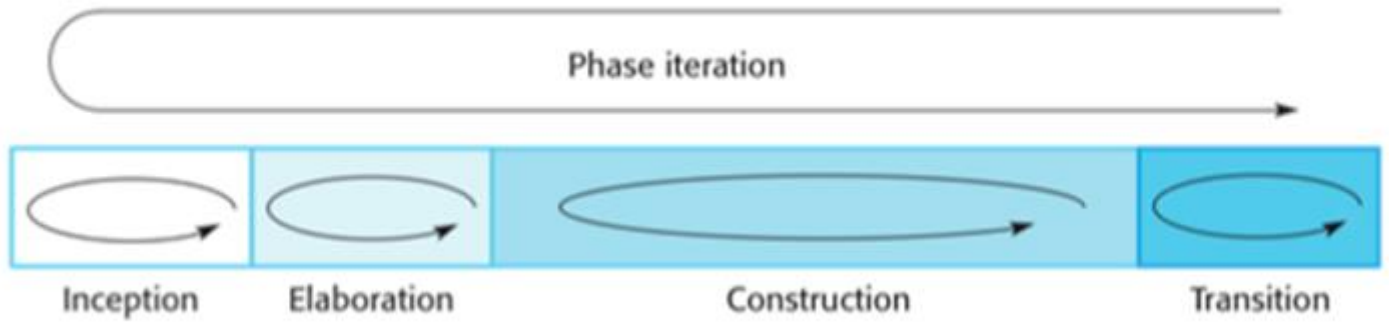
In-Class Activity 02

- Explain the Rational Unified Process (RUP) model elaborating on the following points.
  - Phases of RUP
  - Advantages
  - Disadvantages
  - RUP Best Practices

The Rational Unified Process

- Unified Process (UP) is a popular iterative and incremental software development process framework.
- A modern generic process derived from the work on the UML and associated process.
- Brings together aspects of the 3 generic process models discussed previously.
- Normally described from 3 perspectives
  - A dynamic perspective that shows phases over time;
  - A static perspective that shows process activities;
  - A practive perspective that suggests good practice.

Phases in the Rational Unified Process



RUP phases

- Inception
  - Establish the business case for the system.
- Elaboration
  - Develop an understanding of the problem domain and the system architecture.
- Construction
  - System design, programming and testing.
- Transition
  - Deploy the system in its operating environment.

RUP iteration

- In-phase iteration
  - Each phase is iterative with results developed incrementally.
- Cross-phase iteration
  - As shown by the loop in the RUP model, the whole set of phases may be enacted incrementally.

Static workflows in the Rational Unified Process

Workflow	Description
Business modelling	The business processes are modelled using business use cases.
Requirements	Actors who interact with the system are identified and use cases are developed to model the system requirements.
Analysis and design	A design model is created and documented using architectural models, component models, object models and sequence models.
Implementation	The components in the system are implemented and structured into implementation sub-systems. Automatic code generation from design models helps accelerate this process.

Static workflows in the Rational Unified Process

Workflow	Description
Testing	Testing is an iterative process that is carried out in conjunction with implementation. System testing follows the completion of the implementation.
Deployment	A product release is created, distributed to users and installed in their workplace.
Configuration and change management	This supporting workflow managed changes to the system (see Chapter 25).
Project management	This supporting workflow manages the system development (see Chapters 22 and 23).
Environment	This workflow is concerned with making appropriate software tools available to the software development team.

RUP good practice

- Develop software iteratively
  - Plan increments based on customer priorities and deliver highest priority increments first.
- Manage requirements
  - Explicitly document customer requirements and keep track of changes to these requirements.
- Use component-based architectures
  - Organize the system architecture as a set of reusable components.

RUP good practice

- Visually model software
  - Use graphical UML models to present static and dynamic views of the software.
- Verify software quality
  - Ensure that the software meet’s organizational quality standards.
- Control changes to software
  - Manage software changes using a change management system and configuration management tools.

Thank You!